CHAPTER 5 — SOFTWARE DESIGN

Comparative & Analytical Questions

Section 1: Architectural Foundations & Design Strategies

Q5.C01. Compare Conceptual Design and Technical Design across purpose, audience, and focus.

Dimension	Conceptual Design	Technical Design
Purpose	Describes the system from the customer/user perspective and focuses on what it should do.	Describes how the software is organized and built.
Audience	Customer and users.	System builders.
Focus	External behavior and requirements in domain terms.	Internal architecture and technical construction.
Relation	Designers use the conceptual view to understand and shape the design.	Designers develop the technical view for construction by system builders.

The distinction is about viewpoint and level of technical detail; the source figure does not enumerate all possible documents or validation artifacts.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 204–205; Companion Slide 5, Fig. 2.

Q5.C02. Compare Top-Down, Bottom-Up, and Hybrid Design strategies, and discuss when each may be practical.

Strategy	Starting point	Typical strength	Consideration
Top-Down	System-level functions, refined into lower-level modules.	Keeps the overall system purpose visible.	Lower-level components may not yet be available for early execution.
Bottom-Up	Lower-level components combined into larger subsystems.	Can make use of existing basic components.	System-level organization must still be checked against requirements.
Hybrid	Combines both directions.	Can relate system goals to available components.	The appropriate balance depends on the project.

The chapter treats these as design strategies, not measured rankings of industry popularity. Pure strategies may be impractical for some problems, so a combination can be considered where suitable.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–215; Exercise 5.11.

Q5.C03. Compare Flowcharts and Structure Charts. What design information does each communicate?

Dimension	Flowchart	Structure Chart
Main focus	Procedural steps and decision/control flow.	Module decomposition and calling relationships.
Scope	Describes the logic of a procedure or process.	Describes relationships among modules in a system.
Additional information	Branches and sequence of operations.	Data and control couples passed between modules.

A flowchart is useful for procedural logic; a structure chart is useful for the system's modular hierarchy. They represent different views and should be selected according to the question being modeled.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–218; Companion Slides 41–43; Exercise 5.13.

Section 2: Coupling and Cohesion Dynamics & Trade-Offs

Q5.C04. Explain the design relationship between Module Cohesion and Module Coupling.

Cohesion concerns how strongly the elements within one module belong together. Coupling concerns dependencies between separate modules. A sound modular design aims for high cohesion and low coupling: each module has a focused purpose, while interactions across module boundaries remain limited and clear.

These are complementary goals, not a guaranteed causal law: increasing cohesion in one module does not automatically ensure low coupling elsewhere.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 210–213; Exercise 5.7.

Q5.C05. Why is Logical Cohesion often associated with Control Coupling?

Logical cohesion groups related kinds of activity in one module. If the caller must indicate which activity to perform by passing a selection flag, the module also has control coupling. This is a common design pattern for the example, but not an inevitable consequence of every logically cohesive module.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 208, 211; Exercise 5.8.

Q5.C06. What problems can high coupling create during software maintenance and reuse?

High coupling means modules depend closely on one another. A change to one module may require related changes elsewhere; understanding or reusing a module may also require knowledge of its dependencies. The extent of these effects depends on the particular coupling and architecture.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 207–210; Exercise 5.9.

Q5.C07. What difficulties can low cohesion create?

Low cohesion means that elements within a module have a weak relationship. Such a module may be harder to understand and maintain because its responsibilities are less focused. Changes to one responsibility can also require checking other behavior grouped in the same module.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 210–213; Exercise 5.10.

Q5.C08. Critically evaluate the proposition: "An ideal software architecture should strive for complete decoupling ($Coupling = 0$) between all modules." Is this achievable or desirable?

Critical Architectural Evaluation: The Zero Coupling Fallacy:

Proposition: "*An ideal software architecture should strive for complete decoupling ($Coupling = 0$) between all modules.*"

1. Theoretical and Operational Evaluation: Is Zero Coupling Possible?

No. Complete decoupling in a non-trivial software system is both theoretically and practically impossible.

- **Mathematical and Structural Definition:** A coupling degree of zero ($Coupling = 0$) means that two modules possess absolute zero connectivity: no shared memory, no message passing, no procedural invocation, no parameter transfer, and no shared execution environment.
- **The System Contradiction:** A software system is, by definition, a purposeful collection of collaborating components functioning together to achieve a shared objective. If all modules have zero coupling, they cannot communicate or pass data. The result is not a cohesive software system, but a disjointed collection of isolated programs running in total ignorance of one another.

2. Engineering Desirability: Is Zero Coupling Desirable?

- Striving for zero coupling is a harmful architectural misconception. Software engineering does not seek *zero coupling*, but rather **Loose Coupling (Minimal Necessary Coupling)**.
- Modules must communicate to fulfill business requirements; the goal is to make that communication as clean, narrow, and atomic as possible (specifically, Data Coupling via primitive parameters) while hiding all internal implementation details.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Exercise 5.24, printed p. 249; Companion Slide 15.

Section 3: Object-Oriented vs. Function-Oriented Paradigms

Q5.C09. Compare Abstract Data Types (ADTs) and Object-Oriented Classes using the definitions in the cited sources.

Concept	Description
ADT	A type described through the operations available to clients, while its internal representation is hidden.
Class	A grouping of objects with common characteristics/structure and behavior; it can serve as a blueprint for objects.

These definitions show a useful relationship, but they do not support a blanket claim that every class implements an ADT or that ADTs categorically lack inheritance or polymorphism.

Source: Sommerville (9th ed.), §13.4, PDF p. 356; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 227; Exercise 5.17.

Q5.C10. Evaluate the use of inheritance primarily for code reuse when polymorphic behavior is not needed.

Inheritance can provide reuse, but it also creates a relationship between the subclass and its base class. If the intended relationship between the types is unclear, inheritance may make later changes harder to reason about. Consider whether the relationship models the domain and whether the inherited behavior is appropriate; composition is another option when an object merely needs to use another component.

This is a design evaluation, not a claim that inheritance without overriding is always an anti-pattern.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 228–230; Exercise 5.18.

Q5.C11. Provide an in-depth comparative analysis between Function-Oriented Design (FOD) and Object-Oriented Design (OOD) across paradigms, data ownership, maintenance, and scalability.

Comprehensive Comparative Analysis: FOD vs. OOD:

Analytical Dimension	Function-Oriented Design (FOD / Structured Design)	Object-Oriented Design (OOD)
Guiding Architectural Paradigm	Algorithmic decomposition: System is structured as a hierarchy of functions, procedures, and subroutines.	Object-based decomposition: System is structured as a network of autonomous, interacting objects encapsulating state and behavior.
Primary Modeling Focus	Processes, operations, and functions (Verbs: *calculate*, *sort*, *generate*).	Domain entities, real-world concepts, and abstractions (Nouns: *Customer*, *Account*, *Order*).
Data Ownership & State Management	Data and functions are decoupled. Data is passed across parameter lists or stored in global/shared memory structures.	Data is encapsulated strictly within classes. Direct external mutation is forbidden; access is mediated via validated method contracts.
Decomposition Hierarchy	Top-down functional tree (Structure Charts showing executive coordinator down to operational worker modules).	Distributed web/mesh of collaborating classes connected via associations, aggregation, and inheritance hierarchies.
Handling Requirements Change	Vulnerable: A change in central business data structures ripples across numerous procedural functions that access that data.	Robust: Changes to data representations are contained inside the owning class, leaving client classes isolated from ripple effects.
Extensibility & Polymorphism	Extending functionality requires modifying existing switch/case branches in central coordinator procedures.	Extended naturally by subclassing an existing abstraction and overriding polymorphic methods (Open/Closed Principle).
Unit of Software Reuse	Individual procedural routines or static utility libraries (often limited by hardcoded parameter dependencies).	Cohesive, self-contained classes, design patterns, and polymorphic component frameworks.
Dominant Modeling Notations	Data Flow Diagrams (DFDs), Structure Charts, Data Dictionaries, Mini-specs.	Unified Modeling Language (UML): Class, Sequence, State Machine, Activity, and Component diagrams.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216, 225–226, 246; Companion Slides 36, 58, 92. Exercise 5.22

Section 4: Structural Metrics & Transformation Strategies (Doctor Seminar

Gaps)

Q5.C12. Analyze Fan-In and Fan-Out in software call hierarchies. What are the architectural implications of high fan-in versus high fan-out at top, middle, and leaf layers?

Structural Analysis: Fan-In and Fan-Out Metrics:

Definitions:

- **Fan-Out (Span of Control):** The number of immediate subordinate modules that are directly invoked or controlled by a given module.
- **Fan-In:** The number of distinct superior (calling) modules that directly invoke or utilize a given module.

Architectural Implications & Target Heuristics:

1. **Fan-Out Dynamics (Target: $3 \leq \text{Fan-Out} \leq 7$):**
2. **High Fan-Out at Top/Executive Levels:** Indicates that the coordinating module is attempting to govern too many low-level details directly, lacking intermediate hierarchical decomposition. It increases cognitive complexity and control coupling.
3. **Moderate Fan-Out at Middle Levels:** Reflects well-balanced procedural abstraction, where managers coordinate specialized functional delegates without micromanagement.
4. **High Fan-Out at Leaf Levels:** An architectural violation: leaf modules should be pure workers performing atomic actions without calling multiple subordinates.
5. **Fan-In Dynamics (Target: High Fan-In at Leaf Layers):**
6. **High Fan-In at Bottom/Leaf Layers:** Highly desirable. Indicates that a utility, mathematical, or device-driver module is highly cohesive and widely reused across multiple independent subsystems.
7. **High Fan-In at Top/Middle Layers:** Problematic; indicates that a high-level coordinator is being inappropriately invoked by peer or lower modules, creating cyclic dependencies and inverted architectural layers.

[Top Level] Executive Coordinator (Low Fan-In = 0, Moderate Fan-Out <= 7) | [Middle Level] Subsystem Manager (Moderate Fan-In, Moderate Fan-Out) / \ [Leaf Level] Shared Utilities (High Fan-In >= 5, Fan-Out = 0: Reusable Workers)

Source: Pressman (5th ed.), §13.4.5, printed pp. 347–348 / PDF p. 375; Rajib Mall (4th ed.), §5.4, printed pp. 300–301.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - مقاييس البنية الهرمية (Fan-In و Fan-Out):
> - Fan-Out (مروحة الخروج): عدد الوحدات التابعة التي تستدعيها الوحدة؛ يجب أن يكون معتدلاً (عادة بين 3 و 7) لمنع التعقيد.
> - Fan-In (مروحة الدخول): عدد الوحدات التي تستدعي هذه الوحدة؛ ارتفاعه في الطبقات الدنيا مطلوب وممتاز لأنه دليل على إعادة الاستخدام العالية للوحدات الخدمية (Reusable Utilities).

Q5.C13. Compare Transform Analysis and Transaction Analysis as DFD-to-Structure-Chart mapping strategies. Under what architectural criteria is each strategy selected?

Comparative Analysis: Transform Analysis vs. Transaction Analysis:

Dimension	Transform Analysis (Transform-Centered Design)	Transaction Analysis (Transaction-Centered Design)
Input DFD Flow Characteristics	Linear or sequential data flow: Information enters via input streams, undergoes central mathematical or algorithmic transformation, and exits via output streams.	Branching / Dispatched data flow: An input data item (transaction) arrives and contains a type or opcode that triggers execution of one of several mutually exclusive operational action paths.
Primary Structural Subsystems	1. **Afferent Branch:** Converts raw external inputs into internal logical data forms. 2. **Central Transform:** Performs core algorithmic processing. 3. **Efferent Branch:** Converts internal results into external output formats.	1. **Transaction Reception Branch:** Ingests and validates the incoming transaction token. 2. **Transaction Center (Dispatcher):** Evaluates transaction type and routes control. 3. **Action Paths:** Collection of specialized action modules, one for each transaction type.
Resulting Structure Chart Shape	Balanced three-part tree (Input Controller ← Central Transform Controller → Output Controller).	Broad fan-out architecture rooted at the transaction dispatcher, branching out into mutually exclusive action subtrees.
Typical Application Domains	Scientific computations, digital signal processing, payroll calculation, batch file processing.	ATM terminals, web service request dispatchers, e-commerce checkout systems, command interpreters.

Selection Criteria:

- Select **Transform Analysis** when the system's dominant data flow is an end-to-end pipeline transforming continuous inputs into outputs.
- Select **Transaction Analysis** when the arriving data item dictates which distinct functional subsystem must execute among multiple alternative paths.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 218–220; Companion Slides 42–45; Rajib Mall (4th ed.), §6.4.1, printed pp. 344–349.

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Concepts Covered
1. Architectural Foundations & Strategies	Q5.C01, Q5.C02, Q5.C03	K.K. Aggarwal pp. 204–218; Slides 4–5, 32–42	Conceptual vs technical design, Top-Down vs Bottom-Up vs Hybrid, Flowchart vs Structure Chart.
2. Coupling & Cohesion Dynamics	Q5.C04, Q5.C05, Q5.C06, Q5.C07, Q5.C08	K.K. Aggarwal pp. 207–213, 249; Slides 15–31	Cohesion-coupling duality, logical cohesion induces control coupling, consequences of high coupling and low cohesion, complete decoupling critique.
3. OOP vs FOD Paradigms	Q5.C09, Q5.C10, Q5.C11	K.K. Aggarwal pp. 216, 225–246; Slides 58–92	ADT vs Class, inheritance without polymorphism anti-pattern, comprehensive FOD vs OOD analysis.
4. Structural Metrics & Transformations	Q5.C12, Q5.C13	Pressman §13.4.5; Mall §5.4, §6.4.1	Fan-in and fan-out architectural rules, Transform Analysis vs Transaction Analysis.
